



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

PROJECT TITLE

The domain of the Project:
G37 AI & ML

Team Mentors (and their designation):

Team Members:
Ms. Humera Siddiqua

Period of the project

March 2026 to April 2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Declaration

The project titled “SwipeHire: A Dual Mode AI Driven Job Matching Platform for both Candidates & Recruiters” has been mentored by Mr. Gaurav Patel, organised by SURE Trust, from April 2026 to August 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Ms. Humera Siddiqua

Mr. Gaurav Patel

Data Engineer

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion



Executive Summary

SwipeHire is a full-stack, dual-mode web application built to make online job searching and recruitment faster, more transparent, and more engaging than existing solutions. The platform serves two distinct types of users — job-seeking candidates and hiring recruiters — and gives each of them a tailored experience built around a familiar swipe-based interaction, similar to the kind used in popular consumer apps, but applied here to the serious task of hiring.

The core idea behind SwipeHire is simple: instead of candidates scrolling through long lists of job descriptions and recruiters manually reading through stacks of resumes, both sides are shown one profile or job at a time, along with a clear, calculated compatibility score. Candidates see job postings as swipeable cards, each showing an AI-generated match percentage based on how well their skills align with what the job requires. Recruiters see candidate profiles the same way, ranked by a fit score that combines skill match, resume completeness, and relevant experience. A right swipe means interest; a left swipe means no. When both a candidate and a recruiter swipe right on each other for the same job, the system automatically detects this mutual interest and instantly unlocks a real-time chat window between them, where they can exchange messages, share files such as resumes, and the recruiter can even send a formal interview request — all inside the same platform.

Beyond this core matching loop, the project includes a full set of supporting features that were built to make the platform genuinely usable as a real product rather than a single-feature demo. These include separate multi-step profile setup wizards for candidates and recruiters, editable profiles, a growth analytics dashboard for candidates to track their application activity and top skills over time, a recruiting analytics dashboard for recruiters to see shortlist rates and job performance, a real-time notification system, filters to narrow down the job feed by location and work type, a save-for-later bookmarking option, and a complete administrative panel where a platform admin can manage users, verify recruiter companies, and moderate job postings.

Technically, the entire system was built using the MERN stack — MongoDB for the database, Express.js and Node.js for the backend server, and React.js for the frontend interface — along with Socket.IO to power all the real-time features like chat and instant notifications. The matching logic itself, which is the core "AI" feature of the platform, was built in two stages: it started as a simple, rule-based comparison between candidate skills and job requirements, and was later upgraded to use a real pretrained AI language model that understands the meaning of skills rather than just their exact spelling, allowing it to correctly match skills like "leadership skills" with "leadership qualities" even though the wording is different.

Overall, this project was built as a complete, working, end-to-end application — not just a partial prototype — covering everything from the first screen a user sees when they open the app, all the way through to a scheduled interview between a matched candidate and recruiter.



Introduction

Finding a job and hiring the right person are both processes that, on paper, sound simple, but in practice are often slow, frustrating, and inefficient for everyone involved. On most existing job portals, a candidate has to manually search through job listings using keywords, open each one, read through a long description, and only then figure out whether they're actually a good fit for that role. There is usually no upfront indication of how well their own skills and background match what the employer is looking for. This means candidates often waste a lot of time applying to jobs they were never truly qualified for, or worse, skipping over jobs they would have been a great fit for simply because the listing didn't stand out or wasn't worded the way they searched.

On the other side, recruiters face a similar but mirrored problem. When a job posting goes live, it can receive dozens or even hundreds of applications, and most existing systems present these applications in the order they were received, not in the order of how suitable each candidate actually is. This forces recruiters to manually open and read through resume after resume, spending significant time on candidates who may not even be close to what they're looking for, while potentially strong candidates get buried further down the list simply because they applied later.

Even when a platform does offer some kind of "smart matching" or "recommended for you" feature, it is usually a black box. The candidate has no idea why a certain job was suggested to them, and the recruiter has no idea why one candidate was ranked above another. This lack of transparency makes it hard for either side to trust the system, and it gives them no clear way to understand or improve their own situation — a candidate doesn't know which skill to learn next, and a recruiter doesn't know if the system is actually working correctly.

At the same time, in a completely different space, many popular consumer apps have shown that a simple swipe-based interface — where you're shown one thing at a time and you either say yes or no to it — can dramatically reduce the mental effort of making decisions. People are already comfortable with this style of interaction, and it removes the fatigue of scrolling through long, overwhelming lists.

SwipeHire was built by combining these two ideas. It takes the swipe-based interaction that people already understand and pairs it with a matching system that is honest and transparent about how it calculates its scores. Instead of hiding the logic behind the recommendation, SwipeHire shows the candidate and the recruiter exactly what percentage they matched on, and the project is built so that this score can be explained rather than treated as a mystery. On top of that, the platform doesn't stop the moment a match happens — unlike many similar swipe-based hiring concepts that only handle the discovery step, SwipeHire carries the interaction forward into real-time chat, file sharing, and interview scheduling, so the entire hiring journey can happen within one connected system.



Project Objectives

The project was built with the following clear goals in mind:

To design and build a secure, dual-role authentication system where a single user can sign up as either a candidate or a recruiter, and be guided through a setup process that is specific to their chosen role, so that each type of user only ever sees screens and options that are relevant to them.

To build a genuine skill-matching mechanism that produces a clear, understandable percentage score representing how well a candidate's skills align with a job's requirements, and to make sure this score is calculated in a way that can be explained and defended, rather than being a hidden or unclear number.

To create an interactive, swipe-based interface on both sides of the platform — one for candidates browsing job postings, and one for recruiters reviewing candidate profiles — so that both types of users get a fast, engaging way to make decisions instead of a slow, list-based browsing experience.

To automatically detect when both a candidate and a recruiter have shown interest in each other for the same job, and the moment that happens, to unlock a dedicated communication channel between them without requiring any manual step from either side.

To provide real, working real-time communication features inside that channel, including instant text messaging, the ability to share files such as resumes, and a way for recruiters to formally request an interview directly from within the conversation.

To give both candidates and recruiters a way to look back at their own activity on the platform through dedicated analytics dashboards, so they can track things like how many jobs they've applied to, how many matches they've made, and which skills are most in demand.

To build additional quality-of-life features that a real hiring platform would need, such as the ability to edit a profile after it's first created, filter the job feed by things like location or work mode, and save a job for later without immediately deciding to apply or skip it.

To include a proper administrative layer so that a platform operator can oversee the system as a whole — managing user accounts, verifying which recruiter companies are legitimate, and moderating job postings if needed.

To apply reasonable, real security practices throughout the project, including never storing passwords in plain text, using secure tokens to manage user sessions, and putting basic protections in place against automated abuse of the login system.



Methodology and Results

The project was developed in a structured, stage-by-stage manner, building one working feature completely before moving on to the next, rather than trying to build the entire system all at once. This approach made it possible to test each part of the application thoroughly as it was completed, and to catch and fix problems early rather than letting them pile up.

Development began with the foundational setup of the project, creating two separate parts of the application: a client folder for everything the user sees and interacts with, built using React.js and a build tool called Vite, and a server folder for everything happening behind the scenes, built using Node.js and the Express.js framework. A cloud-hosted database was set up using MongoDB Atlas, which stores all of the platform's data, including user accounts, candidate and recruiter profiles, job postings, matches, and chat messages.

The first major feature built was authentication, allowing users to sign up and log in securely. Passwords are never stored as plain, readable text — instead, they are run through a hashing algorithm called bcrypt before being saved, which scrambles them in a way that cannot be reversed. Once a user logs in successfully, they are given a JSON Web Token, or JWT, which acts like a temporary ID card that proves who they are on every future request they make to the server, without needing to type their password again each time.

After authentication, separate profile-building flows were created for candidates and recruiters. Candidates fill in their skills, education history, work experience, and can upload a resume file. Recruiters fill in their company name, website, industry, and a description of their company, and this company is marked as unverified until an administrator later approves it. Both of these profile-setup processes were later extended so that users could go back and edit their information at any time, rather than being locked into what they entered the first time.

The heart of the project is the matching engine, and this was built and improved in two distinct stages. In the first version, the algorithm worked by taking a candidate's list of skills and a job's list of required skills, cleaning them up so that differences in capitalization or spacing wouldn't cause a mismatch, and then simply counting how many of the job's required skills the candidate actually had. This count was turned into a percentage, which became the match score shown on every job card and candidate card throughout the app. On the recruiter's side, this same match percentage was combined with a small bonus for having a resume uploaded and another small bonus for having relevant work experience, producing what the project calls a "fit score," which is used to automatically rank all the candidates who applied to a job, from strongest to weakest.

In a later stage of development, this matching engine was upgraded to use real artificial intelligence in the form of a pretrained transformer-based language model. Rather than comparing skills as plain text, this newer version converts each skill phrase into a numerical representation of its meaning, known as an embedding, using a model called all-MiniLM-L6-v2. Two skills are then compared using a mathematical formula called cosine similarity, which measures how close their meanings are. This upgrade allows the system to correctly recognize that two differently worded skills, such as "leadership skills" and "leadership qualities," actually mean the same thing, something the earlier, simpler version could not do.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

With the matching logic in place, the actual swipe-based interfaces were built next, using a library called Framer Motion to create smooth, draggable cards that respond to a user's swipe gesture. On the candidate's side, each card represents a job and displays its match percentage; on the recruiter's side, each card represents a candidate and displays their fit score. Swiping right on either side records that decision in the database, and swiping left does the same for a rejection or skip.

To detect when both sides are interested in each other, the server checks, every time someone swipes right, whether the other party has already swiped right on them too, for that same job. If they have, a new match record is automatically created in the database, and this immediately unlocks a chat room specific to that match. Real-time communication was implemented using a technology called Socket.IO, which keeps a live, constantly open connection between a user's browser and the server, allowing chat messages and notifications to appear instantly on screen without the user needing to refresh the page. Inside this chat, users can also share files, and recruiters have a dedicated button to send a formal interview request, which shows up to the candidate as a clear, persistent banner in their conversation.

Once the core matching and communication features were complete, the project was extended with analytics dashboards for both candidates and recruiters, built using a charting library called Recharts, showing things like applications over time, average match percentage, and which skills are most frequently matched. A real-time notification system was added so that users are alerted the moment they get a new match, a new message, or an interview request, shown through a notification bell icon with an unread count. Additional features were then layered on, including the ability to filter the job feed by location, job type, or work mode, a bookmarking feature to save jobs for later review, and a full administrative panel where a platform admin can see overall platform statistics, manage user accounts, verify recruiter companies, and activate or deactivate job postings.

Throughout development, the project was tested manually and thoroughly at each stage, using two separate logged-in sessions at once — one as a candidate and one as a recruiter — to confirm that features like matching, chat, and notifications worked correctly and instantly between two real, independent accounts. The result of this process is a fully functioning, end-to-end platform where every feature described in the project's objectives has been implemented, tested, and confirmed to work as intended.



Social / Industry Relevance of the Project

The problems that SwipeHire is designed to solve are not abstract or invented — they reflect real, everyday frustrations experienced by both job seekers and employers. For a candidate, especially someone early in their career or searching for their first job, the process of applying to dozens of positions with little to no feedback on why they weren't selected can be genuinely discouraging. A platform that immediately shows a candidate a clear, understandable match percentage gives them useful, actionable information — they can see at a glance which jobs are worth their time, and by understanding how the score is calculated, they can even identify which skills might be worth learning to improve their chances in the future.

For recruiters, particularly those working at smaller companies or startups that cannot afford expensive, enterprise-level hiring software, the ability to have applicants automatically ranked by a fit score can save a significant amount of time and effort. Instead of manually reading through every single resume in the order it arrived, a recruiter can focus their attention on the candidates the system has identified as the strongest matches first, while still being able to review everyone if they choose to.

The decision to make the matching process transparent, rather than hiding it behind a black-box algorithm, also has broader relevance in today's world, where there is increasing public and industry concern about AI systems making decisions that affect people's lives — including hiring decisions — without those people understanding why. By building a system where the match score can always be explained in terms of exactly which skills matched and which didn't, this project reflects a growing and important industry trend toward explainable, trustworthy AI, rather than opaque automation.

Additionally, the technical approach used in this project — taking an existing, pretrained AI model and applying it to a new problem, rather than attempting to train a brand-new model from scratch — is directly representative of how AI is actually built and deployed in most real-world companies today. This method, known as transfer learning, is both realistic and practical, and demonstrates an industry-relevant skill: knowing how to responsibly and effectively use existing AI tools rather than assuming every AI feature requires building a model from the ground up.

Finally, by connecting the matching process directly to real-time communication and interview scheduling, the project addresses a genuine gap that exists even in some of the more polished hiring products currently on the market — many of these tools help you find a match, but then leave you to move the conversation to email, phone calls, or other external tools. Keeping the entire journey inside one connected platform, from first swipe to scheduled interview, is a meaningful and practical improvement that has clear value for real users on both sides of the hiring process.



Learning and Reflection

Building this project from start to finish involved a significant amount of hands-on learning that went well beyond simply following a tutorial or copying existing code. Setting up a full-stack application meant learning how to properly structure a project so that the frontend and backend could communicate cleanly, how to design a database schema in MongoDB that could handle the different, sometimes complex shapes of data needed for candidates, recruiters, jobs, and matches, and how to build a secure authentication system using industry-standard tools like bcrypt for password hashing and JWT for session management, rather than attempting to invent a custom, less secure approach.

One of the more challenging and educational parts of the project was implementing the real-time features using Socket.IO. Unlike the rest of the application, which follows a simple request-and-response pattern where the browser asks for something and the server sends back an answer, real-time chat and notifications required learning an entirely different way of thinking, where the server and browser keep an open connection and can send messages to each other at any time, without either side needing to ask first. Understanding concepts like rooms, where users are grouped so that messages only go to the right people, was an important and genuinely new piece of technical knowledge gained during this project.

A particularly valuable and honest learning experience came from the process of upgrading the matching algorithm from a simple, rule-based comparison to a real AI-powered version using a pretrained language model. This transition required understanding the practical difference between a function that runs instantly and one that requires asynchronous processing, since AI model inference takes a small but real amount of time to complete. Making this change meant carefully going back through the existing codebase and updating every place that called the matching function, adding the proper handling needed for asynchronous code, which reinforced a genuinely important lesson in software development: that changing how one core piece of a system works can have ripple effects throughout the rest of the codebase, and those effects need to be tracked down and handled carefully rather than assumed to be automatic.

The debugging process throughout the project was, in many ways, just as educational as the feature-building itself. Encountering and eventually resolving issues such as CSS class names accidentally conflicting between different components, subtle bugs caused by comparing database ID objects incorrectly, and syntax errors introduced through copy-pasting code, all helped build a more careful, methodical approach to identifying exactly where a problem was coming from, rather than guessing or making broad, unfocused changes.

Finally, this project prompted an honest and important reflection on what the term "AI" actually means in a technical sense, versus how it is often used loosely in marketing and everyday conversation. Starting with a simple, rule-based algorithm and clearly understanding that it was not genuine machine learning, and then later deliberately upgrading to a real, pretrained neural network model, made the distinction between automation and true AI much clearer, and reinforced the importance of being able to honestly and accurately describe exactly what a system does, rather than overstating its capabilities.



Conclusion and Future Scope

While the current version of SwipeHire is a complete, fully working platform, there are several meaningful directions in which it could be extended further. The matching algorithm could be enhanced to allow recruiters to mark certain required skills as absolutely essential versus simply preferred, giving the match score a more nuanced and realistic weighting rather than treating every required skill as equally important. As the platform gains more real users and more swipe history builds up over time, the current approach could also be extended into a hybrid system that considers not just skill similarity, but also patterns learned from which types of matches have historically been successful.

On the practical side, adding email notifications alongside the existing real-time, in-app alerts would help ensure users don't miss important updates when they aren't actively using the platform. Automatically generating a video call link the moment a recruiter sends an interview request, rather than requiring that to be arranged manually afterward, would further streamline the path from match to actual interview. A lightweight skill-verification feature, where candidates could complete short assessments to earn a verified badge next to a specific skill, would also strengthen the overall trustworthiness of the platform by grounding the AI match score in demonstrated ability rather than self-reported claims alone.

In conclusion, this project set out to address genuine, well-understood problems in the online recruitment process — inefficient discovery, a lack of transparency in matching, and a disconnected communication process after a match is made — and succeeded in building a complete, working solution to all three. By combining a familiar, low-friction swipe-based interface with a matching engine that is honest and explainable about how it calculates compatibility, and by carrying that match all the way through to real-time chat, file sharing, and interview scheduling within a single platform, SwipeHire demonstrates that meaningful, practical improvements to the hiring process are achievable without requiring overly complex or opaque technology. The project also reflects a realistic and responsible approach to using artificial intelligence, choosing tools and techniques appropriate to the scale and purpose of the system, and remaining honest throughout about exactly what those tools do and do not do.